BSDA2001P: Introduction to DL and GenAI

Project Report

Genre Classification from Messy Mashup Audio

Name: Abhishek Kumar
Student ID: 21f2000477
Competition: Kaggle — Jan 2026 DL GenAI Project
Best Kaggle Score: 0.91082 (Public Leaderboard)

1. Abstract / Executive Summary

This report documents a solution to the Kaggle 'Messy Mashup' music genre classification competition. The task is to classify noisy audio mashups into one of ten genres. The central challenge is a distribution mismatch — training data consists of clean separated audio stems, while test data consists of noisy real-world mashups.

A progressive approach was taken: classical ML (LR, NB, XGBoost on MFCC), custom deep learning (CNN, CRNN on Mel Spectrograms), and a fine-tuned Audio Spectrogram Transformer (AST) with synthetic mashup training and confidence-weighted Test-Time Augmentation (TTA). Key results:

Model	Val Macro F1	Kaggle Score
Logistic Regression (MFCC)	0.5882	0.13350
XGBoost (MFCC)	0.6412	0.07754
Custom CNN (Mel Spectrogram)	0.6081	0.25278
AST Basic (pretrained)	0.9068	0.77447
AST + Synthetic Mashup + TTA	**0.9565**	**0.91082**

Core insight: bridging the train-test distribution gap through synthetic mashup generation and noise augmentation yielded far greater gains than any architectural change alone.

2. Introduction

The Kaggle competition provides clean separated audio stems (drums, bass, vocals, other) as training data across 10 genres, while test data consists of noisy mashup files mixed with environmental noise. Models must generalise from clean stems to noisy, mixed audio — a significant domain adaptation challenge.

Objectives: (1) Establish classical ML baselines with MFCC features; (2) Build custom CNN and CRNN deep learning models on Mel Spectrograms; (3) Fine-tune a pretrained AST transformer; (4) Address the train-test gap through synthetic mashup generation and ESC-50 noise augmentation; (5) Apply confidence-weighted TTA at inference.

3. Dataset, EDA & Preprocessing

3.1 Dataset

Source: jan-2026-dl-gen-ai-project / messy_mashup (Kaggle). Contains: genres_stems/ (10 genre folders, each with song subfolders containing drums.wav, bass.wav, vocals.wav, other.wav), mashups/ (noisy test files), test.csv, and ESC-50-master/ (2000 environmental noise .wav files, ~5s each).

3.2 Exploratory Data Analysis

Figure 1 — Songs Per Genre

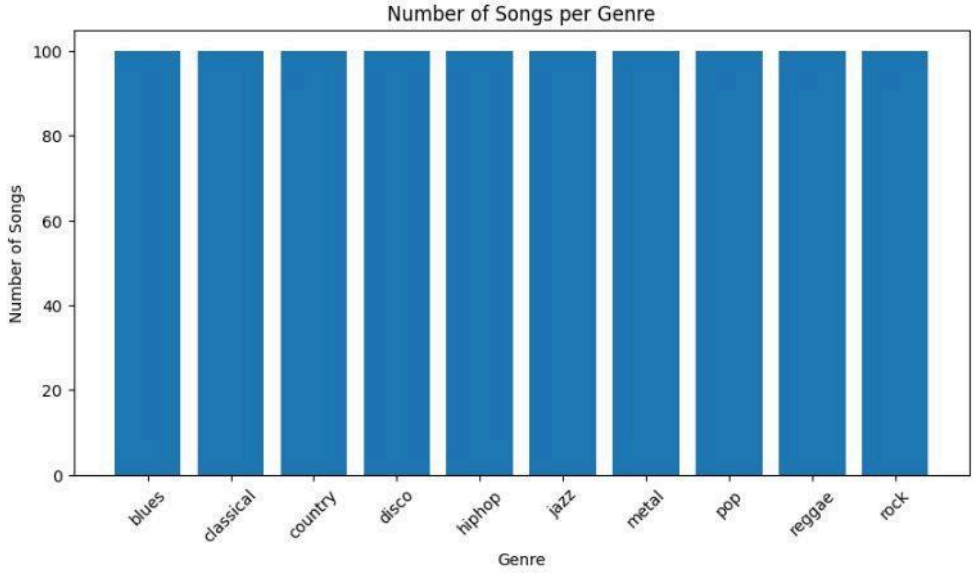

Figure 1: Bar chart — Number of Songs per Genre

Insight: Each of the 10 genres contains exactly 100 songs, confirming a perfectly balanced dataset. This eliminates the need for class-weighting or oversampling strategies during training. A balanced distribution also means Macro F1 and accuracy are equally informative metrics.

Figure 2 — Audio Duration Distribution

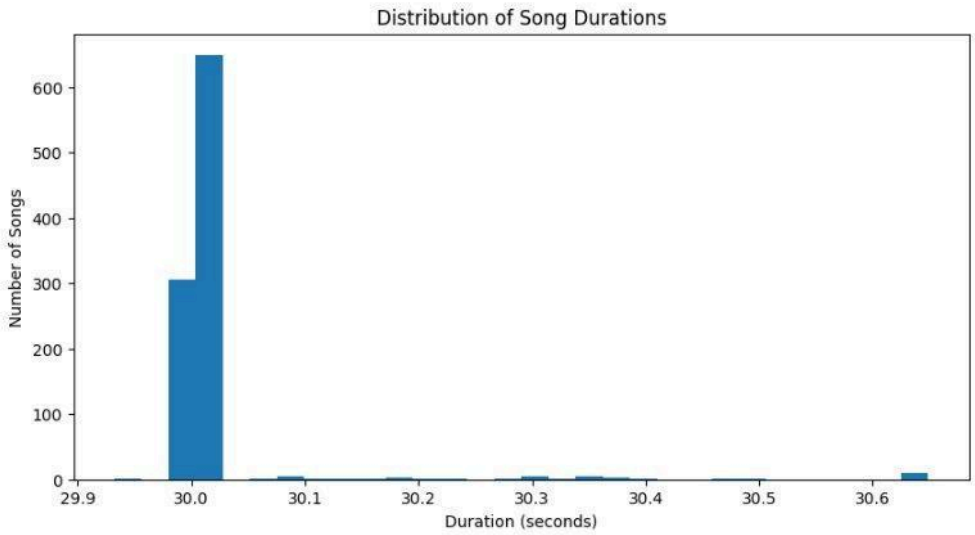

Figure 2: Histogram — Distribution of Song Durations (seconds)

Insight: Almost all songs cluster tightly at ~30 seconds, with the vast majority between 30.0–30.05s. A tiny fraction extends slightly beyond 30.6s. This consistent duration simplifies preprocessing — a fixed 30s crop is applied for CNN/CRNN models and a random 10s crop is used for AST training, with minimal padding needed.

Figure 3 — MFCC Visualization

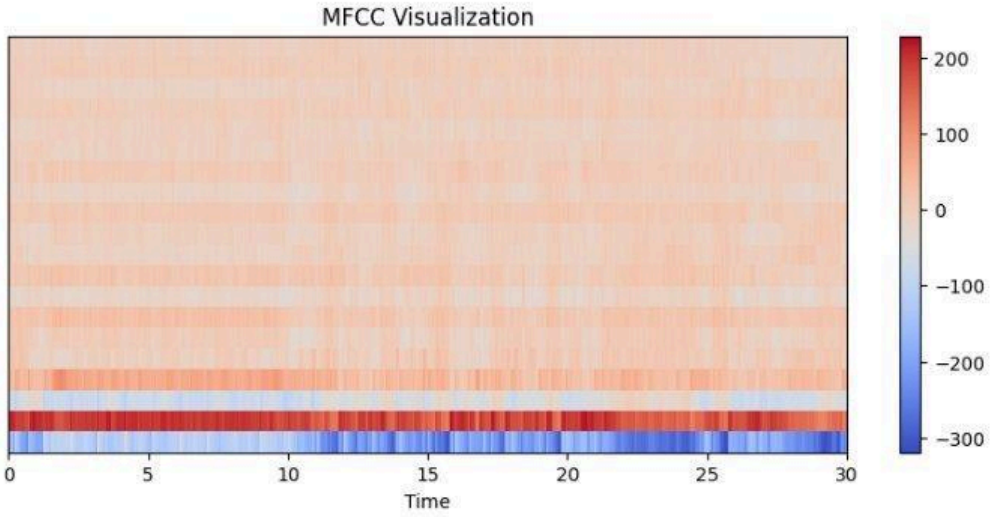

Figure 3: MFCC Heatmap — rock.00000.wav (20 coefficients × time frames)

Insight: The MFCC heatmap shows the 20 cepstral coefficients (y-axis) across time (x-axis). Lower coefficients (top rows, warm orange/red) capture broad spectral shape and energy — these are relatively stable and genre-informative. Higher coefficients (bottom rows, cold blue) encode fine timbral details that are noisier. The mean of each coefficient across time was used as the feature vector for classical ML — this collapses the temporal axis and loses dynamic variation, which explains why MFCC-based models struggled on noisy mashups.

Figure 4 — Mel Spectrogram: Rock

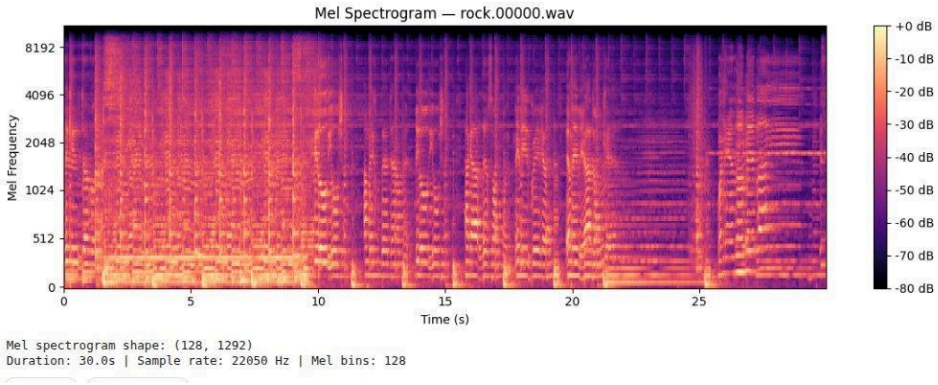

Figure 4: Mel Spectrogram — rock.00000.wav (128 Mel bins, 30s, SR=22050 Hz)

Insight: The rock spectrogram shows strong broadband energy across all frequency bins (bright orange/yellow regions spread from low to high Mel frequencies). The rhythmic vertical striping visible around 0–2048 Hz corresponds to drum hits and guitar transients. High-frequency energy (4096–8192 Hz) is consistently present — characteristic of electric guitar distortion and cymbal crashes common in rock. These rich spectral patterns serve as the input to CNN/CRNN models.

3.3 Preprocessing Pipeline

Classical ML: Four stems per song summed into a combined waveform → 20 MFCC coefficients extracted → mean across time → 20-D feature vector. Labels encoded with LabelEncoder. 80/20 stratified split.

CNN/CRNN: Combined songs loaded at SR=22050 Hz, padded/cropped to 30s → 128-bin log Mel Spectrogram in dB → saved as .npy. DataLoader batch_size=16.

AST models: Audio resampled to 16000 Hz. Random 10s crop per sample during training. ESC-50 noise added on-the-fly. Audio preloaded into RAM cache for fast throughput.

3.4 Data Augmentation — Synthetic Mashup Generation

The most impactful preprocessing step. The create_synthetic_mashup() function picks a random stem from a different song within the same genre for each of the 4 stem types, sums them, and normalises the result. This generates within-genre mashups that directly match the test data structure.

300 synthetic mashups were generated per genre → 3000 total training samples. Combined with ESC-50 noise augmentation (alpha randomly sampled from 0.1 to 0.4), this directly bridges the distribution gap.

4. Feature Extraction Strategy

Feature	Used For	Shape	Key Strength
MFCC mean (20 coeff.)	LR, NB, XGBoost	20-D vector	Fast, compact
Mel Spectrogram (128 bins)	CNN, CRNN	(1, 128, T)	2D spectral-temporal
AST patch spectrogram	AST (all variants)	Patch sequence	Full attention + AudioSet prior

Primary extractor: ASTFeatureExtractor (MIT/ast-finetuned-audioset-10-10-0.4593). Converts raw 16kHz audio to a 2D spectrogram divided into patches — analogous to tokens in NLP. Applied on-the-fly inside ASTDataset.__getitem__() enabling dynamic cropping and augmentation. Justified because the pretrained AudioSet vocabulary gives an extremely strong prior over diverse real-world audio.

5. Modeling & Experimentation

Seven models were trained across three paradigms. All deep learning runs were tracked with Weights & Biases (wandb).

5.1 Model 1 — Logistic Regression (Baseline)

Features: 20-D MFCC mean vector. LogisticRegression(max_iter=500), one-vs-rest. Training time: 0.01 min. Val Macro F1: 0.5882. Kaggle: 0.13350. Simple linear baseline — confirms MFCC alone insufficient for noisy audio.

5.2 Model 2 — Naive Bayes

GaussianNB on the same 20-D MFCC features. Training time: 0.00 min. Val Macro F1: 0.3478. Weakest model — independence assumption poorly suited to correlated MFCC coefficients.

5.3 Model 3 — XGBoost on MFCC

XGBClassifier(n_estimators=300, max_depth=6, learning_rate=0.05). Best classical ML model. Val Macro F1: 0.6412. Kaggle: 0.07754. Despite better val F1 than LR, the Kaggle score was worse — strong evidence that MFCC features trained on clean stems generalise very poorly to noisy mashups.

5.4 Model 4 — Custom CNN on Mel Spectrograms

Architecture

```
Input: (1, 128, T) — Mel Spectrogram
  │
  ├─ Conv2d(1→16, 3×3, pad=1) → ReLU → MaxPool2d(2,2)
  ├─ Conv2d(16→32, 3×3, pad=1) → ReLU → MaxPool2d(2,2)
  ├─ AdaptiveAvgPool2d(8, 8)
  ├─ Flatten → Linear(2048, 128) → ReLU
  └─ Linear(128, 10) → Output (10 genres)
```

Training: CrossEntropyLoss, Adam(lr=0.001), batch_size=16, 10 epochs. AdaptiveAvgPool ensures fixed output regardless of input length. Logged to wandb ('CNN' run). Best model saved by val F1.

Results: Train F1 ~0.8249, Val F1 ~0.6081, Kaggle: 0.25278. Significant overfitting (train-val gap of ~0.22) — model memorised clean song patterns, failed on noisy mashups. Motivated CRNN experiment.

5.5 Model 5 — Custom CRNN (CNN + Bidirectional LSTM)

Architecture

```
Input: (1, 128, T) — Mel Spectrogram
  │
  ├─ Conv2d(1→32) → BN → ReLU → MaxPool2d(2,2)
  ├─ Conv2d(32→64) → BN → ReLU → MaxPool2d(2,2)
  ├─ Conv2d(64→128) → BN → ReLU → MaxPool2d(2,2)
  ├─ Mean across freq dim → Permute to (B, T, 128)
  ├─ BiLSTM(input=128, hidden=128, layers=2, dropout=0.3)
  ├─ Mean pool over time → Dropout(0.3)
  └─ Linear(256, 10) → Output (10 genres)
```

Training: Adam(lr=0.001, weight_decay=1e-5), gradient clipping (max_norm=5), 10 epochs. Logged to wandb ('CRNN' run).

Results: Best Val Macro F1: 0.5687. Final epoch — Train F1: 0.5659, Val F1: 0.4646. Training time: 1.12 min. BiLSTM over Mel time-frames did not add benefit — worse than CNN. Moved to pretrained transformer.

5.6 Model 6 — AST Basic (Pretrained Fine-Tuning)

Architecture

```
ASTForAudioClassification (MIT/ast-finetuned-audioset-10-10-0.4593)
  │
  ├─ ASTFeatureExtractor → 2D Spectrogram Patches
  ├─ Patch Embedding + Positional Embedding
  ├─ 12× Transformer Encoder Blocks
  │     (Multi-Head Self-Attention + FFN + LayerNorm)
  ├─ [CLS] token → Classification Head
  └─ Linear(768, 10) → Output (10 genres)  [replaced head]
```

Fine-tuning: All layers unfrozen. Adam(lr=2e-5), CrossEntropyLoss, batch_size=8, 5 epochs. Trained on combined clean songs with ESC-50 noise augmentation. Logged to wandb ('AST-pretrained' run).

Results: Best Val F1: 0.9068. Training time: 23.25 min. Kaggle: 0.77447. The gap between val F1 (0.9068) and Kaggle (0.77447) confirmed the distribution problem persists even with a powerful pretrained model.

5.7 Model 7 — Final AST + Synthetic Mashup Training + TTA

Architecture & Key Improvements

Same AST architecture as Model 6, with three targeted improvements to address distribution mismatch:

```
Training Data:  3000 synthetic mashups (300/genre × 10 genres)
                + ESC-50 noise augmentation (alpha ∈ [0.1, 0.4])
                + Random 10s crop per sample

Model:          ASTForAudioClassification (same as Model 6)
Optimizer:      AdamW(lr=2e-5)  |  Loss: CrossEntropyLoss
Epochs: 5       Batch: 8        Training time: 62.09 min

Inference (TTA):
  for each test file:
    crops = 16 overlapping 10s windows
    for each crop:
      logits = AST(crop)
      conf   = max(softmax(logits))
      weighted_logits += logits × conf
    prediction = argmax(weighted_logits / sum(conf))
```

Hyperparameter Details

Parameter	Value / Justification
Learning rate	2e-5 — small enough for stable fine-tuning without catastrophic forgetting
Batch size	8 — limited by GPU memory with AST's large spectrogram inputs
Epochs	5 — val F1 peaked at epoch 3 (0.9583); further epochs gave diminishing returns
TTA crops	16 — balances inference time (~85 min) vs robustness
Noise alpha	uniform(0.1, 0.4) — mild to moderate noise matching expected test conditions
Mashups per genre	300 — practical limit given Kaggle storage and generation time

Epoch-by-Epoch Results

Epoch	Val Macro F1	Notes
1	0.9127	Strong start — synthetic data helps immediately
2	0.8950	Slight dip — typical mid-training variance
3	0.9583 ← Best	Model saved at this checkpoint
4	0.9355	Minor drop
5	0.9565	Strong finish

Best Val Macro F1: 0.9565 | Kaggle Public Score: 0.91082 | Inference time: ~85+ min

6. Performance & Comparative Analysis

6.1 Complete Model Comparison

Model	Feature	Val F1	Kaggle	Train Time
Random Baseline	—	~0.10	—	<1s
Logistic Regression	MFCC	0.5882	0.13350	0.01 min
Naive Bayes	MFCC	0.3478	—	0.00 min
XGBoost	MFCC	0.6412	0.07754	0.05 min
Custom CNN	Mel Spec.	0.6081	0.25278	~few min
Custom CRNN	Mel Spec.	0.5687	—	1.12 min
AST Basic	AST patches	0.9068	0.77447	23.25 min
AST + Synthetic + TTA	**AST patches**	**0.9565**	**0.91082**	**62.09 min**

6.2 WandB Training Logs

Figure 6 — Validation Macro F1 Across All Models

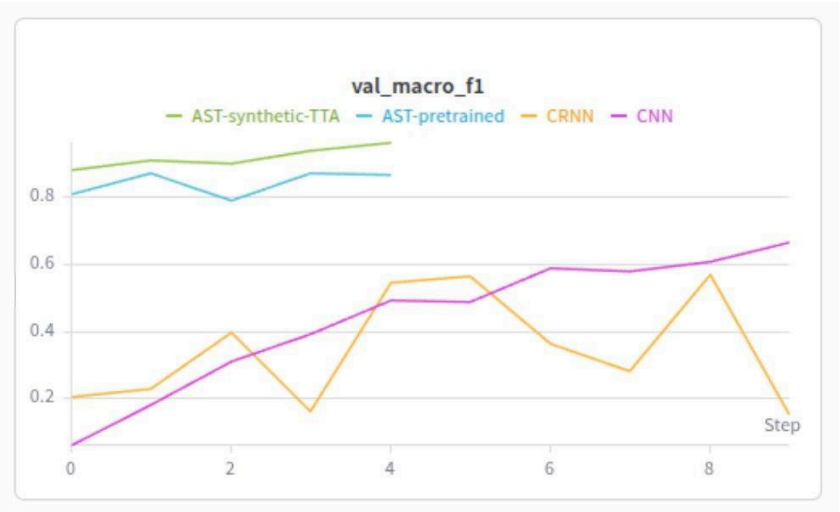

Figure 6: WandB — val_macro_f1 per epoch (AST-synthetic-TTA, AST-pretrained, CRNN, CNN)

Insight: The chart clearly shows four distinct performance tiers. The two AST models (green and blue) consistently dominate, both staying above 0.80 throughout training. AST-synthetic-TTA (green) is higher and more stable, starting at ~0.87 and climbing to ~0.93, confirming that synthetic mashup training immediately helps from epoch 1. AST-pretrained (blue) plateaus around 0.85-0.86. In sharp contrast, CNN (pink) starts near 0 and slowly climbs to ~0.66, while CRNN (orange) oscillates erratically between 0.15 and 0.55, never converging reliably. The gap between the transformer-based and custom models is dramatic — over 0.25 F1 points — demonstrating the decisive advantage of AudioSet pretraining.

Figure 7 — Training Loss Across All Models

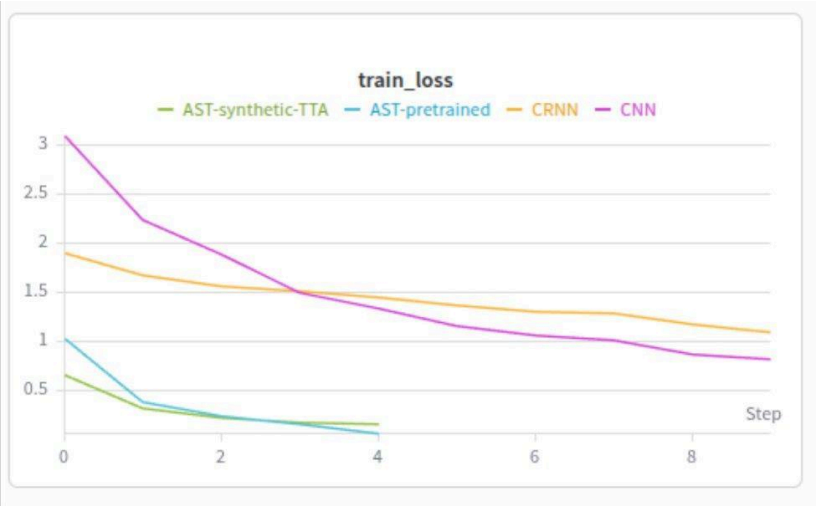

Figure 7: WandB — train_loss per epoch (AST-synthetic-TTA, AST-pretrained, CRNN, CNN)

Insight: The training loss chart reveals stark differences in learning efficiency. Both AST models (green and cyan) start with low initial loss (~0.6 and ~1.0 respectively) and converge rapidly to near-zero by epoch 2-3, reflecting the strong AudioSet pretraining that requires very little gradient descent to adapt. CNN (pink) starts very high (~3.0) and declines slowly over 10 epochs, reaching ~0.8 — still much higher than the AST models after 5 epochs. CRNN (orange) also starts high (~1.9) but decreases less smoothly, settling around ~1.1 with visible instability. The fast convergence of AST models is not just due to architecture — the synthetic mashup training data provides much more relevant gradients from the first batch, explaining why AST-synthetic-TTA (green) has an even lower starting loss than AST-pretrained (cyan).

6.3 Error Analysis

Key failure patterns observed across models:

- Classical models on noisy mashups: XGBoost val F1 0.6412 collapsed to Kaggle 0.07754 — MFCC mean features carry almost no signal on noisy mashups. Temporal mean pooling collapses all dynamic variation, making the 20-D vector nearly meaningless under noise.
- CNN overfitting: Train F1 0.82 vs Val F1 0.61 — the model memorised spectral fingerprints of clean stems (e.g., specific drum patterns common in rock) rather than genre-level abstractions. On noisy mashups, these patterns were drowned out.
- CRNN temporal failure: BiLSTM processed the Mel time axis sequentially, but genre-discriminative features are distributed non-locally. Global attention of transformers is more effective.
- AST Basic Kaggle gap (0.9068 val → 0.77447 Kaggle): Even the pretrained AST fine-tuned on clean songs struggled on noisy mashups — it had never seen stems mixed with environmental noise during training.
- Final AST remaining errors: Likely from mashups where genre signal is very weak — e.g., very short clips, or mashups where 3 of 4 stems come from a minority genre. Confidence-weighted TTA helps but cannot fully recover when all crops are ambiguous.

6.4 Key Insights

- Distribution gap dominates: Validation F1 is a poor proxy for Kaggle score. The clean-to-noisy gap must be explicitly addressed in training.
- Synthetic data > architecture: The biggest jump (0.77447 → 0.91082) came from training data design, not model changes.
- Confidence-weighted TTA is effective: Weighting by model certainty reduces variance on ambiguous audio.
- Attention > recurrence for audio: AST's global self-attention dramatically outperformed CRNN's sequential BiLSTM.

7. Conclusion & Future Work

7.1 Key Learnings

- Addressing distribution shift directly is the highest-leverage strategy — more impactful than any architectural change.
- Pretrained audio transformers (AST on AudioSet) dramatically outperform classical ML and custom DL for noisy audio classification.
- MFCC mean features are too low-dimensional — temporal collapse discards critical discriminative information.
- WandB logging was essential for identifying overfitting and comparing models across runs.

7.2 Challenges & Future Improvements

- Long inference time (85+ min for TTA) limited submission iterations. Future: batch inference or fewer crops.
- Larger synthetic dataset (500–1000/genre) and cross-genre mashup training for greater diversity.
- Advanced augmentation: pitch shifting, time stretching, room impulse response convolution.
- Larger pretrained models: BEATs or larger AST variants could yield higher scores.

8. References

[1] 3Blue1Brown. (2024). Neural Networks [YouTube Playlist]. https://www.youtube.com/playlist?list=PL-wATfeyAMNoirN4idjev6aRu8ISZYVWm

[2] Valerio Velardo — The Sound of AI. (2021). Audio for Machine Learning [YouTube Playlist]. https://www.youtube.com/playlist?list=PL-wATfeyAMNqIee7cH3q1bh4QJFAaeNv0

[3] Hugging Face. (2024). Audio Spectrogram Transformer (AST) — Model Documentation. https://huggingface.co/docs/transformers/en/model_doc/audio-spectrogram-transformer

[4] Wandb. (2020). Weights & Biases: Developer tools for ML. https://wandb.ai

[5] Kaggle Competition: Jan 2026 DL GenAI Project. https://www.kaggle.com/competitions/jan-2026-dl-gen-ai-project